AOSP 进阶丛书 · 第二册 · ANDROID 13 (API 33)

# 系统启动主线逐函数精读

按下电源键到 Launcher 出现的每一步：两阶段 init 与 rc 触发瀑布、单线程事件驱动主循环、Zygote 预加载与 fork、SystemServer 三阶段服务总装与 boot phase 时间线——把开机变成白盒。

---

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

# 目 录

前言：把开机变成你的白盒 . . . . . 2
第一章 启动全景：九个阶段 . . . . . 2
第二章 init 诞生：main() 的四个身份 . . . . . 3
第三章 第二阶段 init：SecondStageMain 逐段精读 . . . . . 5
第四章 init 语言：rc 文件的解析与求值 . . . . . 7
第五章 init 主循环：单线程事件驱动 . . . . . 15
第六章 Zygote：App 世界的祖先 . . . . . 16
第七章 SystemServer：三阶段服务总装 . . . . . 21
第八章 开机完成：systemReady 到 BOOT_COMPLETED . . . . . 25
第九章 启动性能：主线上的优化战场 . . . . . 29
第十章 设计决策十问 . . . . . 29
第十一章 深水区：bootloader、fastboot 协议与 OTA 引擎 . . . . . 30
附录 A 观测实操手册 . . . . . 31
附录 B 关键文件清单（本册出场人物） . . . . . 32
附录 C 自检题 . . . . . 33
附录 D 动手实验（一天量） . . . . . 33
教科书习题集（分级） . . . . . 33
术语表（按拼音排序） . . . . . 35
排障手册：启动问题定位 . . . . . 36

**本册速览**

九阶段总时序：BootROM→init 两阶段→Zygote→SystemServer→BOOT_COMPLETED
rc 触发瀑布与单线程事件循环 · Zygote 预加载与 fork 流水线
boot phase 100→1000 时间线 · sys.boot_completed 权威标志
深水区：bootloader 两级跳 · fastboot 协议 · GKI · update_engine
边读边做：dmesg/bootchart 对位九阶段，附录 D 五个实验

# 前言：把开机变成你的白盒

第一册给了你地图，本册带你沿主线行军：**按下电源键到 Launcher 出现**的每一步，都落到可验证的函数与行号上。这一条线串起了 init、rc 语言、属性系统、Zygote、ART、SystemServer、AMS——它们是 Android 系统开发的“文言文”，读懂它，后面所有子系统的阅读速度都会质变。

本册源码基线与第一册相同（AOSP Android 13，`$A` 代指源码树根）。建议配合真机或模拟器同步实验：`dmesg`、`logcat -b events` 里的每一行，本册都会告诉你它从哪个源文件打出。

# 第一章 启动全景：九个阶段

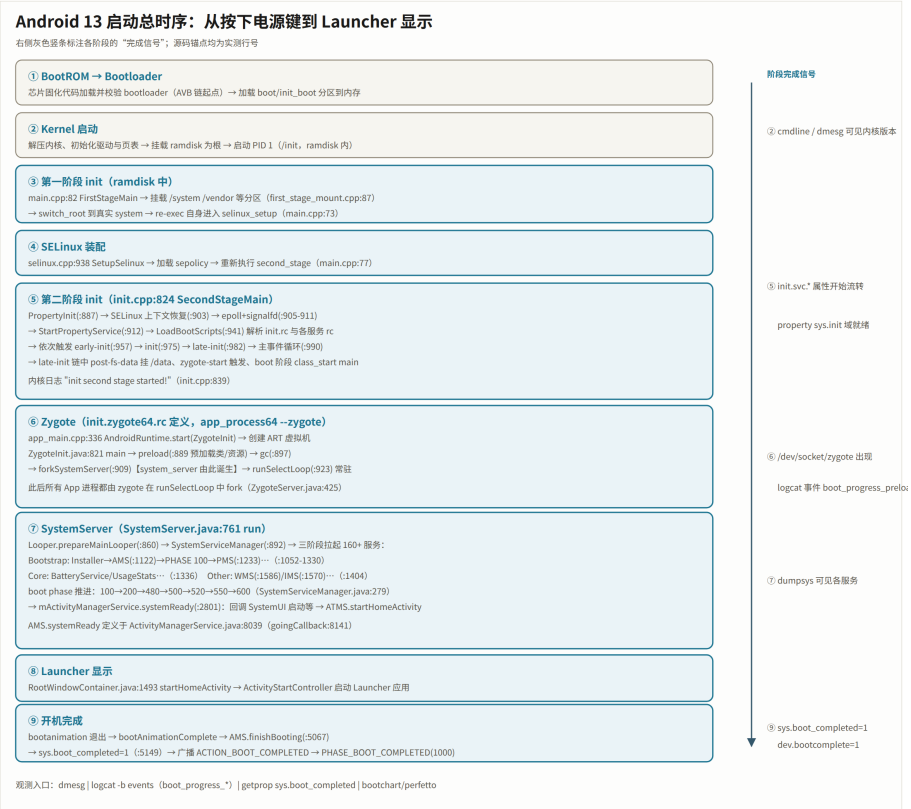



图 1 Android 13 启动总时序：BootROM 到 BOOT_COMPLETED 的九个阶段与各阶段完成信号

九个阶段里有三个分水岭值得先记住：

- **内核 → 用户态**：PID 1（init）诞生，此后的世界全部由 init 直接或间接孵化。
- **native → Java**：Zygote 创建 ART 虚拟机并预加载框架，此后 Framework 是 Java 的世界。
- **就绪 → 可用**：`AMS.systemReady()` 之后才允许启动 Home 与三方应用，`sys.boot_completed=1` 是全机“开机完成”的唯一权威标志（`ActivityManagerService.java:5149`）。

时间量级参考（中端机）：BootROM+bootloader 数十毫秒，内核一两秒，init 两阶段数百毫秒，zygote 预加载数百毫秒到一秒，SystemServer 两三秒。**启动优化的战场几乎全在后三段**——也就是本册第二到第九章覆盖的范围。

# 第二章 init 诞生：main() 的四个身份

init 的可执行文件只有一个（/system/bin/init），但它以四种身份被反复执行。入口在 system/core/init/main.cpp：

cpp system/core/init/main.cpp:53-82（节选）

```
int main(int argc, char** argv) {
// 讲解注释①：优先级 -20（最高），保证早期初始化独占 CPU；
// 到主循环前恢复 0（:989）——"特权只在必要时持有"。
setpriority(PRIO_PROCESS, 0, -20);
// 讲解注释②：身份判据是 argv[0]——同一个二进制被 symlink 成 ueventd 时，
// 内核 exec 传来的 argv[0] 不同，init 借此"变身"（省一个 ELF 的空间）。
if (!strcmp(basename(argv[0]), "ueventd")) {
return ueventd_main(argc, argv); // 身份①：ueventd
}
if (argc > 1) {
if (!strcmp(argv[1], "subcontext")) { ... } // 身份②：SELinux 沙箱子进程
if (!strcmp(argv[1], "selinux_setup")) {
return SetupSelinux(argv); // 身份③：装载 SELinux
}
if (!strcmp(argv[1], "second_stage")) {
return SecondStageMain(argc, argv); // 身份④：第二阶段主体
}
}
// 讲解注释③：兜底身份=第一阶段 init。ramdisk 里的 /init 被内核拉起时
// 不带参数，走到这里；此后两次 re-exec 才依次切换到③④身份。
return FirstStageMain(argc, argv);
}
```

> **设计洞察 · 逐段精读：一个二进制的四次人生**
> ①argv[0]=ueventd（设备节点产房）；②subcontext（SELinux 沙箱里的 builtin 执行器）；③selinux_setup（装配策略后 re-exec）；④second_stage（真正的主角）。判断身份的手段全是**进程自己的属性**（argv/环境变量），不依赖任何外部状态——这是"最早期代码只能相信自己"的生存法则。

**一码多身**是精心设计：ramdisk 空间寸土寸金，一个二进制兼任 init/ueventd/subcontext 避免了多份 libc/ELF；身份切换只靠 argv——Unix 世界最古老的开关。

## 2.1 第一阶段：挂上真实的根

FirstStageMain（first_stage_init.cpp:218）只做一件事：**把散落的分区拼成一个可启动的根文件系统**。

- 从设备树读 fstab（first_stage_mount.cpp:79 的 FirstStageMount，:87 DoFirstStageMount），挂载 /system、/vendor 等；
- PrepareSwitchRoot（:121）与 SwitchRoot("/first_stage_ramdisk")（:397）：把根从 ramdisk 切到真实 system 分区；

- 检查 /force_debuggable（:371）支持解锁调试 ramdisk；
- 最后 **re-exec 自身**，参数 selinux_setup——因为装载 SELinux 策略需要“重启自身”来应用新的安全上下文。

## 2.2 SELinux 装配

SetupSelinux（selinux.cpp:938）加载 /system/etc/selinux/precompiled_sepolicy 等策略文件、恢复关键文件标签，然后再次 re-exec 进入 second_stage。至此系统完成了“先立规矩（安全策略）再办事”的前奏——**任何服务进程被拉起之前，它的 SELinux 域就已注定**。

## 2.3 并行伙伴：ueventd 与设备节点

main.cpp:61 的 argv[0] 分支让 init 的同一二进制兼任 ueventd（system/core/init/ueventd.cpp）。它在 early-init 前被拉起，监听内核 netlink 的 uevent，按 ueventd.rc 与 ueventd.${ro.hardware}.rc 规则创建 /dev/* 设备节点并设置权限——**所以 /dev 的世界里没有 udev，只有 ueventd**。主流程的 wait_for_coldboot_done（init.cpp:960）等它完成冷插扫描才继续，这解释了为什么两者的顺序是“ueventd 先跑、init 等、再触发”。firmware 加载（firmware_handler.cpp）也在 ueventd 侧：驱动请求固件时它从 /vendor/firmware 等路径搬运文件进内核。

> **设计洞察 · 为什么分两阶段**
> 第一阶段运行在 ramdisk 的临时根上，而 SELinux 策略、rc 剧本、服务二进制都在 /system 分区——不先挂分区就无事可做；而挂载动作本身又不该受旧 SELinux 状态污染。两阶段 + 两次 re-exec，用最朴素的方式解决了“鸡生蛋”：每个阶段只信任它已经亲手布置好的世界。

## 2.3.1 ueventd 深度走读：/dev 世界的缔造者

ueventd 与 init 是**同一个二进制**的不同身份（main.cpp:61 argv[0] 切换）。它在 early-init 之前被拉起，专职监听内核 uevent 并创建设备节点。

### ueventd 的主循环

cpp system/core/init/ueventd.cpp（骨架+讲解注释）

```
int ueventd_main(int argc, char** argv) {
// 讲解注释①：解析 ueventd.rc 和 ueventd.${ro.hardware}.rc
// 规则格式：/dev/<name> <mode> <owner> <group>
// 例：/dev/graphics/* 0660 system graphics
// 讲解注释②：注册 netlink socket 监听内核 uevent
// 每个设备插拔/驱动注册 → 内核发 uevent → ueventd 收到
// 讲解注释③：冷启动扫描（coldboot）：
// 首次启动时主动扫描 /sys/class/* 下所有已注册设备
// → 为每个设备创建对应的 /dev 节点
// 讲解注释④：热插拔监听：
// USB 插入/拔出 → 内核发 uevent → ueventd 创建/删除节点
// → 触发 init 的 trigger（如 usb插入触发 vold 启动）
```

```
}
```

### 设备节点创建的完整链

text 从内核驱动到 /dev 节点

```
内核驱动加载（如 touch driver）
→ driver 注册 platform_device
→ 内核 kobject_uevent() 发 NETLINK_KOBJECT_UEVENT
→ ueventd 的 socket 收到消息
→ 解析 uevent（ACTION=add, DEVNAME=input/event3, SUBSYSTEM=input）
→ 查 ueventd.rc 匹配规则（权限/属主）
→ mknod("/dev/input/event3", mode, dev_t)
→ chmod + chown 按规则设置
→ 此后 App/framework 才能打开这个设备
// 讲解注释⑤：主流程 wait_for_coldboot_done（init.cpp:960）
// init 等 ueventd 完成冷启动扫描才继续触发后续 action——
// 因为 surfaceflinger 要打开 /dev/graphics/fb0，
// 没有设备节点=服务起不来
```

### 常见 ueventd 排障

| 症状 | 原因 | 排查 |
| --- | --- | --- |
| 某设备节点不存在 | ueventd 规则缺失或冷启动漏扫 | cat /proc/bus/input/devices |
| 节点权限错 | ueventd.rc 规则错误 | ls -la /dev/<name> 对比预期 |
| USB 热插拔无反应 | ueventd 死或 SELinux 拦截 | logcat -s ueventd; getenforce |

> **设计洞察 · 逐段精读：为什么不用 udev**
>
> Linux 桌面用 udev（一个功能更丰富的守护进程），Android 选了更简单的 ueventd——**启动阶段越少依赖越好**。ueventd 与 init 同二进制意味着零额外空间成本、共享 SELinux 策略、共享日志通道。这是"极简主义"的工程设计典范。

# 第三章 第二阶段 init：SecondStageMain 逐段精读

SecondStageMain（init.cpp:824）是用户态世界的真正起点。以下按源码顺序分段精读（重要行号全部实测）。

## 3.1 环境自保（:824-886）

cpp system/core/init/init.cpp:824-886（节选）

```
int SecondStageMain(int argc, char** argv) {
if (REBOOT_BOOTLOADER_ON_PANIC) InstallRebootSignalHandlers();
boot_clock::time_point start_time = boot_clock::now();
SetStdioToDevNull(argv); // stdio 全部指向 /dev/null
InitKernelLogging(argv); // 日志改走 /dev/kmsg
LOG(INFO) << "init second stage started!";
```

```
...
struct sigaction action = {.sa_flags = SA_RESTART};
action.sa_handler = [](int) {}; // SIGPIPE 置为 no-op（:851-855）
sigaction(SIGPIPE, &action, nullptr);
...
WriteFile("/proc/1/oom_score_adj", ...); // init 永不被 LMK 杀（:858-863）
keyctl_get_keyring_ID(KEY_SPEC_SESSION_KEYRING, 1); // 会话密钥环（FBE 加密钥匙落点）:868
close(open("/dev/.booting", O_WRONLY | O_CREAT | O_CLOEXEC, 0000)); // “正在开机”哨兵 :871
```

这些琐碎动作各有一句“潜台词”：stdio 归零是因为子服务会继承 fd，泄漏一个 console fd 就等于开一个后门；SIGPIPE 不能杀 init，否则任何子进程退出都可能带走 PID 1；`/dev/.booting` 哨兵告知固件加载器“稍安勿躁”。**系统级代码的第一素养：每个“为什么”都有安全或时序答案。**

## 3.2 基础设施三件套（:887-930）

cpp system/core/init/init.cpp:887-941（节选）

```
PropertyInit(); // ① 属性区与默认值 :887
...
SelinuxSetupKernelLogging(); // ② SELinux 日志通道 :901
SelabelInitialize(); // 加载 file_contexts :902
SelinuxRestoreContext(); // 递归恢复全盘标签 :903
Epoll epoll;
epoll.Open(); // ③ 事件中枢 :905-908
InstallSignalFdHandler(&epoll); // SIGCHLD → signalfd（不嵌信号处理器）:910
InstallInitNotifier(&epoll);
StartPropertyService(&property_fd); // 属性写仲裁 socket :912
...
SetupMountNamespaces(); // 每个服务可获独立 mount ns :932
ActionManager& am = ActionManager::GetInstance();
ServiceList& sm = ServiceList::GetInstance();
LoadBootScripts(am, sm); // 解析全部 .rc :941
```

三个设计细节值得放大：

- **signalfd 而非 signal handler**：信号处理器会在任意线程任意字节码中间打断执行，是数据竞争之源；signalfd 把 SIGCHLD 变成 epoll 里的一个普通 fd，与 init 单线程模型严丝合缝（:829-832 的注释明言 signalfd 之前禁止起线程）。
- **属性先于一切服务**：后面所有 rc 的 `on property:` 触发器都依赖属性区存在。
- **LoadBootScripts 的搜索路径**（:307-322）：`/system/etc/init/hw/init.rc` → `/system/etc/init` → `/system_ext/etc/init` → `/vendor/etc/init` → `/odm/etc/init`——**分区边界即 rc 边界**，vendor 的服务只在 vendor 的 rc 里声明。

## 3.4 属性仲裁：handle_property_set_fd

cpp system/core/init/property_service.cpp:527-604（节选）

```
static void handle_property_set_fd() {
int s = accept4(property_set_fd, nullptr, nullptr, SOCK_CLOEXEC);
...
struct ucred cr; // 对端凭证：内核背书的 pid/uid
socklen_t cr_size = sizeof(cr);
```

```
getsockopt(s, SOL_SOCKET, SO_PEERCRED, &cr, &cr_size);
...
if (mac_len > 0) { // SELinux 上下文随请求携带
char* source_ctx = ...;
uint32_t result = HandlePropertySet(prop_name, prop_value,
source_ctx, cr, &socket, &error);
```

SO_PEERCRED 是 Unix 域套接字的“身份证读取器”——**凭证由内核填写，客户端无法伪造**。这与 Binder 驱动填充 sender_euid（第三册 IPCThreadState.cpp:1270）是同一个安全哲学的两处实现：身份必须在内核边界采集。

### 3.3 点火：触发器入队（:953-986）

cpp system/core/init/init.cpp:953-986（节选）

```
am.QueueBuiltinAction(SetupCgroupsAction, "SetupCgroups");
...
am.QueueEventTrigger("early-init"); // :957
am.QueueBuiltinAction(wait_for_coldboot_done_action, ...); // :960 等 ueventd 冷插
...
am.QueueEventTrigger("init"); // :975
std::string bootmode = GetProperty("ro.bootmode", "");
if (bootmode == "charger") {
am.QueueEventTrigger("charger"); // 充电模式：不开机只充电 :979
} else {
am.QueueEventTrigger("late-init"); // :982 正常开机主干
}
am.QueueBuiltinAction(queue_property_triggers_action, ...); // :986
```

注意 charger 分支：插着充电器开机时 init 走完全不同的剧本——不启动 zygote、不进 boot 阶段，只点亮充电界面。**同一个 init，两套人格，全靠触发器分流**。

# 第四章 init 语言：rc 文件的解析与求值

### 4.1 剧本总谱：late-init 瀑布

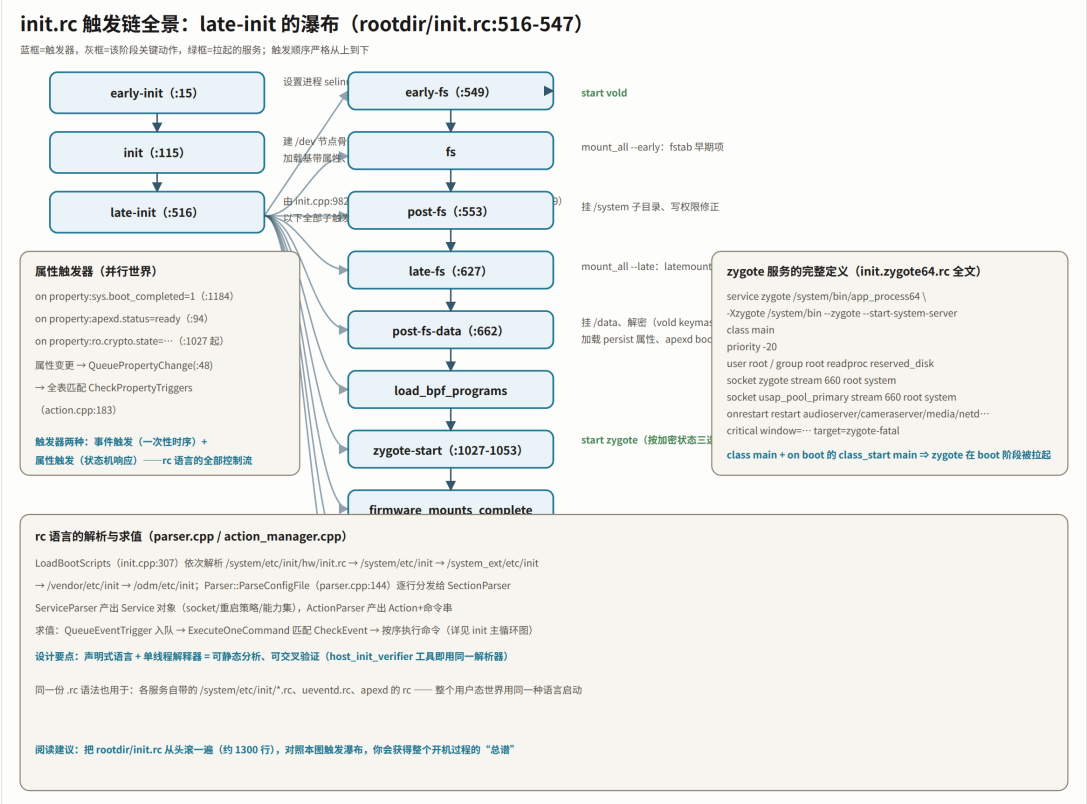



图 2 init.rc 触发链全景：late-init 的瀑布式子触发器、属性触发器并行世界与 zygote 服务完整定义

`rootdir/init.rc` 的 `on late-init`（:516-547）段是全剧总谱，子触发器严格按书写顺序入队：`early-fs → fs → post-fs → late-fs → post-fs-data → load_bpf_programs → zygote-start → firmware_mounts_complete → early-boot → boot`。每个名字都对应“文件系统与关键服务可用性”的一级台阶，`zygote-start` 之前必须完成 /data 解密（`post-fs-data`），因为应用数据与 dex 都在 /data 上。

zygote 的启动条件用三个属性触发器表达（`:1027-1053`）：`ro.crypto.state=unencrypted / unsupported / encrypted&&file`——**加密状态决定 zygote 何时可以跑**，这就是“顺序依赖”在声明式语言里的表达方式。

## 4.2 解析器三件套

`Parser::ParseConfigFile`（`parser.cpp:144`）读文件后按第一个 token 分发：`service` 交给 `ServiceParser`（构造 `Service` 对象：二进制路径、class、socket、重启策略、SELinux 域、能力集），`on` 交给 `ActionParser`（构造 `Action` + 命令串），`import` 递归解析。同一套解析器还服务于 `host_init_verifier`（构建期校验 rc 的正确性）——**一套语法，三处使用（开机求值、静态校验、CTS 检查）**，这是声明式设计换来的红利。

## 4.3 Action 求值：一次一条

cpp system/core/init/action_manager.cpp:67-94（节选）

```
void ActionManager::ExecuteOneCommand() {
{
```

```
auto lock = std::lock_guard{event_queue_lock_};
while (current_executing_actions_.empty() && !event_queue_.empty()) {
for (const auto& action : actions_) {
if (std::visit([&action](const auto& event) {
return action->CheckEvent(event); }, event_queue_.front())) {
current_executing_actions_.emplace(action.get());
}
}
event_queue_.pop();
}
}
...
action->ExecuteOneCommand(current_command_); // 只执行队首 action 的第 N 条命令
++current_command_;
```

一次主循环只推进**一条命令**。这意味着一段 50 条命令的 action 会与其他触发器交错执行——没有谁能霸占 init。代价是吞吐，但 init 的 KPI 从来不是吞吐，是**确定性**。

## 4.4 服务孵化：do_class_start 与 Service::Start

`on boot` 里的 `class_start core/main` 走到 `builtins.cpp:163`：

cpp system/core/init/builtins.cpp:163-176（节选）

```
static Result<void> do_class_start(const BuiltinArguments& args) {
for (const auto& service : ServiceList::GetInstance()) {
if (service->classnames().count(args[1])) {
if (auto result = service->StartIfNotDisabled(); !result.ok()) { ... }
}
}
return {};
}
```

`Service::Start`（`service.cpp:524`）是所有守护进程的产房：先建 socket（`:613`，所以 zygote 的 `/dev/socket/zygote` 在 fork 前就存在），再 `fork()`（`:633`），子进程 `RunService` 后 `exec` 目标二进制（`:638`）；父进程登记 pid、写 oom_score_adj（`:647`）、创建 cgroup（`:663`）、`NotifyStateChange("running")`（`:684`）——**把 `init.svc.zygote` 属性置为 running**。属性即状态机，`getprop init.svc.zygote` 因此成为运维常识。

## 4.5 rc 语言语法要素速查

| 语句 | 语义 | 示例 | 求值时机 |
| --- | --- | --- | --- |
| service | 声明服务及其属性 | service zygote /system/bin/app_process64 … | 解析期建对象，class_start 时启动 |
| on <trigger> | 声明动作段 | on boot / on property:sys.boot_completed=1 | 触发器命中后逐命令执行 |
| trigger <name> | 排队子触发器 | trigger zygote-start（写在 late-init 内） | 作为命令入队 |

| 语句 | 语义 | 示例 | 求值时机 |
| --- | --- | --- | --- |
| import | 引入其他 rc | import /init.${ro.zygote}.rc | 解析期展开 |
| class_start 等 builtin | 命令 | class_start main / mount / setprop / exec | Action 求值时执行 |
| onrestart / ondestruct | 服务挂钩 | onrestart restart audioserver | 服务重启时执行 |
| critical / oneshot / disabled | 服务策略 | critical window=... target=zygote-fatal | Reap 时决定重启行为 |

语言细节最能体现“声明式”的地方：`service` 段里**没有任何控制流**，属性就是断言；`on` 段的命令序列是纯顺序；一切分支由触发器与属性在运行期决定。这也是 rc 文件能被 `host_init_verifier` 在构建机上完整校验的原因——不存在“跑到才知道”的行为。

## 4.6 init.rc 完整语言参考（教科书级速查）

### service 段全部属性

| 属性 | 语法 | 含义 | 示例 |
| --- | --- | --- | --- |
| name + path | service X /path/to/bin | 服务名+可执行路径 | service zygote /system/bin/app_process64 |
| class | class <name> | 启动类别（class_start 批量启动） | class main |
| user / group | user <u> group <g> | 运行身份 | user root group root readproc |
| socket | socket <n> <t> <p> <o> <g> | 创建 Unix socket | socket zygote stream 660 root system |
| oneshot | oneshot | 只跑一次不重启 | -- |
| onrestart | onrestart <cmd> | 重启时执行 | onrestart restart audioserver |
| critical | critical | 死则整机重启 | critical window=... target=... |
| disabled | disabled | 不随 class_start 启动 | -- |
| seclabel | seclabel <label> | SELinux 域标签 | seclabel u:r:zygote:s0 |
| priority | priority <n> | 进程优先级 | priority -20 |
| task_profiles | task_profiles <p> | cgroup 任务档 | task_profiles ProcessCapacityHigh |

| 属性 | 语法 | 含义 | 示例 |
| --- | --- | --- | --- |
| interface | interface <name> | HAL 服务声明 | interface android.hardware.power.IPower/default |
| writepid | writepid <file> | 写 pid 到文件（watchdog 用） | -- |

## on 段全部命令（常用 20 个）

| 命令 | 语法 | 含义 |
| --- | --- | --- |
| mkdir | mkdir <path> [mode] [owner] [group] | 创建目录 |
| mount | mount <type> <dev> <dir> [flag...] | 挂载文件系统 |
| chmod / chown | chmod <mode> <path> | 改权限/属主 |
| setprop | setprop <name> <value> | 设属性 |
| start / stop / restart | start <svc> | 控制服务 |
| class_start | class_start <class> | 批量启动 |
| exec | exec [<seclabel>] [--] <cmd> <args> | 执行外部命令（阻塞） |
| wait | wait <path> [timeout] | 等待文件出现 |
| copy | copy <src> <dst> | 复制文件 |
| write | write <path> <content> | 写文件 |
| symlink | symlink <target> <link> | 创建符号链接 |
| trigger | trigger <event> | 排队子触发器 |
| verity_update_state | verity_update_state <mount> | 更新 dm-verity 状态 |

## import 语句

text import 的解析顺序（LoadBootScripts :307）

```
/system/etc/init/hw/init.rc ← 总谱
/system/etc/init/*.rc ← 各服务自带 rc
/system_ext/etc/init/*.rc ← 扩展分区
/vendor/etc/init/*.rc ← vendor 分区
/odm/etc/init/*.rc ← odm 分区
// 讲解注释：分区边界即 rc 边界——vendor 的服务只在 vendor 的 rc 里声明
```

**设计洞察 · 逐段精读：rc 语言的设计哲学**

**声明式**（无控制流，属性即断言）+ **触发器**（事件驱动+属性驱动）+ **单一解析器**（开机求值/静态校验/CTS 检查三处复用）。这三个设计决策让 init.rc 保持了 15 年的稳定性——**好的 DSL 是"少即是多"的教科书**。

# 9.1 启动性能分析实战：用 bootchart 定位慢在哪

## bootchart 是什么

bootchart 是 init 内建的启动时间线采集器（system/core/init/bootchart.cpp）：设置 `persist.init.bootchart` 后，init 记录各 action/service 的启动时间戳，配合 `grabbootchart.sh` 产出甘特图。

## 五步实操

bash bootchart 实操五步（可直接执行）

```
# 步骤 1：开启采集
adb shell setprop persist.init.bootchart 1
# 步骤 2：重启
adb reboot
# 步骤 3：等待开机完成
adb wait-for-device shell 'while [ "$(getprop sys.boot_completed)" != "1" ]; do sleep 1;
done'
# 步骤 4：停止并导出
adb shell setprop persist.init.bootchart stop
adb pull /data/bootchart /tmp/bootchart
# 步骤 5：生成甘特图
cd /tmp/bootchart && tar -czf bootchart.tgz *
python3 system/core/init/grabbootchart.py /tmp/bootchart
```

## 典型 bootchart 输出解读

text bootchart 甘特图（简化示意）

```
时间(ms) 0 500 1000 1500 2000 2500 3000 4000
init ████████████ ← init 本体
ueventd ██████ ← 设备节点
vold ████████ ← 卷管理
surfaceflinger ████████████████ ← 图形
zygote ████████████████████████ ← preload 最耗时
system_server ████████████████████████████████ ← 最多服务
bootanimation ████████████████████████████████← 动画直到 BOOT_COMPLETED
// 讲解注释①：zygote 段长的原因：preloadClasses（3000+ 类）
// → 优化方向：裁剪 preloaded-classes（Go 版走 --enable-lazy-preload）
// 讲解注释②：system_server 段长的原因：PMS 扫描全机 APK
// → 优化方向：并行扫描（SystemServerInitThreadPool）
// 讲解注释③：bootanimation 到 BOOT_COMPLETED 才停
// → 优化方向：提前放行动画（AMS.systemReady 不必等全部服务）
```

## 常见优化手段及效果

| 优化 | 预期收益 | 风险 |
| --- | --- | --- |
| 裁剪 preloaded-classes | -500ms~1s | 首帧变慢（类未预载） |
| lazy preload（Go 版默认） | -300ms~800ms | 首个 App 启动变慢 |
| PMS 并行扫描 | -500ms~2s | 稳定性（部分设备不稳定） |

| 优化 | 预期收益 | 风险 |
| --- | --- | --- |
| dex2oat 提前到 OTA 后 | -1s~3s | OTA 后首次开机变慢 |
| SystemServer 线程池并行初始化 | -300ms~1s | 需要梳理依赖 |
| 冻结 bootanimation 帧 | 省 0ms（体验优化） | -- |

**设计洞察 · 逐段精读：性能优化的方法论**

**度量先行**（bootchart 定位）→ **找瓶颈**（zygote preload？PMS 扫描？system_server？）→ **定向优化**（不要盲目调）→ **复验**（重跑 bootchart 对比）。这个"度量→分析→优化→验证"闭环是所有系统性能工作的通用方法论——比记住任何单个数字都值钱。

**源码锚点 · 本章锚点速查**

<font face="MonoFT" size="9">parser.cpp:144</font> ParseConfigFile / <font face="MonoFT" size="9">:238</font> ParseConfig 目录入口<br/><font face="MonoFT" size="9">action_manager.cpp:43</font> QueueEventTrigger / <font face="MonoFT" size="9">:57</font> QueueBuiltinAction / <font face="MonoFT" size="9">:67</font> ExecuteOneCommand<br/><font face="MonoFT" size="9">builtins.cpp:163</font> do_class_start；命令表 <font face="MonoFT" size="9">:1387</font> 起<br/><font face="MonoFT" size="9">service.cpp:524</font> Start / <font face="MonoFT" size="9">:633</font> fork / <font face="MonoFT" size="9">:684</font> NotifyStateChange / <font face="MonoFT" size="9">:258</font> Reap<br/><font face="MonoFT" size="9">rootdir/init.rc:516-547</font> late-init 瀑布 / <font face="MonoFT" size="9">:1027</font> zygote-start 三条件

## 4.7 解析器核心走读：从文本到内存对象

init.rc 是纯文本文件——解析器把它变成内存中的 Service/Action 对象。这段代码是**声明式 DSL 解析的教科书**。

### Parser 主流程

cpp system/core/init/parser.cpp（骨架+逐行注释）

```
bool Parser::ParseConfigFile(const std::string& path) {
// 讲解注释①：读取文件内容到内存
auto config_contents = ReadFile(path);
if (!config_contents.ok()) return false;
// 讲解注释②：逐行解析（ParseData → ParseLine）
// 每行按第一个 token 分发：
// "service" → ServiceParser（下一节）
// "on" → ActionParser
// "import" → 递归调用 ParseConfig
ParseData(path, &config_contents.value());
return true;
}
// ParseLine 的分发逻辑
void Parser::ParseLine(std::vector<std::string>&& args, const std::string& filename, int
line) {
```

```
if (args[0] == "service") {
// 讲解注释③：ServiceParser::ParseSection
// args[1]=服务名，args[2]=可执行路径
// 后续行（class/socket/user等）→ ParseLineSection
service_parser_->ParseSection(std::move(args), filename, line);
} else if (args[0] == "on") {
// 讲解注释④：ActionParser::ParseSection
// args[1..]=触发条件（trigger 或 property:name=value）
// 后续行（命令）→ ParseLineSection 追加到 Action
} else if (args[0] == "import") {
// 讲解注释⑤：递归解析（支持变量展开 ${ro.hardware}）
ParseConfig(args[1]);
}
}
```

## Service 对象的完整生命周期

text rc → Service 对象 → fork → 运行 → 死亡 → 重启

```
rc 文件：Service 对象：OS 进程：
"service zygote ┌──────────────────┐
/system/bin/app_process64 │ name_: "zygote" │
--zygote │ pathname_: "..." │
class main" │ classnames_:{main}│
│ sockets_: [zygote]│
│ onrestart_: [...] │
│ pid_: 0（未启动）│
└────────┬─────────┘
│ class_start main
▼
┌──────────────────┐ fork+exec
│ Service::Start() │──────────────→ [zygote 进程]
│ socket 创建 │
│ pid_ = fork() │
│ SVC_RUNNING │
└────────┬─────────┘
│ SIGCHLD（进程退出）
▼
┌──────────────────┐
│ Service::Reap() │
│ 决定是否重启 │
│ critical→重启整机 │
│ oneshot→不重启 │
│ 默认→延迟重启 │
└──────────────────┘
```

## 一个 service 段被解析后的内存状态

text 解析 init.zygote64.rc 后的 Service 对象

```
Service {
name_: "zygote"
pathname_: "/system/bin/app_process64"
args_: ["-Xzygote", "/system/bin", "--zygote", "--start-system-server"]
classnames_: {"main"} // class_start main 触发
sockets_: [{name:"zygote", type:"stream", perm:660, uid:"root", gid:"system"}]
```

```
onrestart_: [
exec_background - system system -- /system/bin/vdc volume abort_fuse
write /sys/power/state on
restart audioserver
restart cameraserver
restart media
...
]
priority_: -20
seclabel_: "u:r:zygote:s0"
flags_: SVC_CRITICAL | SVC_RESTARTEXITING
pid_: (运行时填充)
}
```

**设计洞察 · 逐段精读：解析器设计的三个精妙之处**

①**同一解析器三处使用**：开机求值、构建期校验（host_init_verifier）、CTS 检查——声明式 DSL 的复用价值。②**延迟启动**：解析=建对象，class_start=真正 fork——语义与执行分离。③**变量展开在解析期**：${ro.hardware} 在 ParseConfig 时就展开（不是运行时）——保证确定性。

# 第五章 init 主循环：单线程事件驱动




图 3 init 主循环：epoll 事件源、ActionManager 事件队列、命令执行与服务生命周期，附单线程设计动机

主循环（`init.cpp:990-1056`）的骨架只有五步：查关机命令 → `am.ExecuteOneCommand()`（推进一条命令）→ `HandleProcessActions()`（服务重启调度，并算出下次唤醒时间）→ `epoll.Wait(timeout)` → 处理就绪 fd（**先 reap 子进程再处理其他事件**，`:1025-1028` 注释解释了原因

：防止“服务已死但还没收尸，ctl.start 请求却被拒”的竞态）。

**为什么单线程**（图右下详述）：无锁可推理、逐命令公平、行为确定性可度量。对照第三册你会看到 system_server 的 Binder 线程池世界——两种模型各自服务“顺序的世界”与“并发的世界”。

## 5.1 主循环源码精读

cpp system/core/init/init.cpp:990-1031（节选）

```
while (true) {
auto epoll_timeout = std::optional<std::chrono::milliseconds>{kDiagnosticTimeout};
auto shutdown_command = shutdown_state.CheckShutdown();
if (shutdown_command) { HandlePowerctlMessage(*shutdown_command); } // 关机指令
if (!(prop_waiter_state.MightBeWaiting() || Service::is_exec_service_running())) {
am.ExecuteOneCommand(); // ① 每轮只推一条命令
}
if (!IsShuttingDown()) {
auto next_process_action_time = HandleProcessActions(); // ② 服务重启调度
if (next_process_action_time) {
epoll_timeout = ceil<milliseconds>(*next_process_action_time - now());
}
}
...
auto pending_functions = epoll.Wait(epoll_timeout); // ③ 等事件/定时
...
ReapAnyOutstandingChildren(); // ④ 先收尸（:1028）
for (const auto& function : *pending_functions) { (*function)(); } // ⑤ 再处理 fd
HandleControlMessages(); // ⑥ ctl.start/stop 请求
}
```

三个容易被忽略的精妙处：`prop_waiter_state.MightBeWaiting()`——属性等待（`wait_for_prop` 命令）期间暂停推进命令，保证测试脚本时序；`HandleProcessActions` 返回“下次需要醒来的时刻”，epoll 的超时因此**精确到服务重启节拍**，循环空转率为零；`exec` 服务（如 `exec u:r:vendor_modem:s0 ...`）执行期间主循环只等 SIGCHLD 并在超时后输出诊断（`:1032-1050`）——单线程模型对阻塞命令的补偿。

# 第六章 Zygote：App 世界的祖先

## 6.1 出生：app_process 的使命




图 4 Zygote 的出生、预加载、forkSystemServer 与 runSelectLoop 孵化流水线，附 8 条设计动机

init 按 `init.zygote64.rc` 拉起 `/system/bin/app_process64 -Xzygote /system/bin --zygote --start-system-server`。`app_main.cpp` 解析参数后：

cpp frameworks/base/cmds/app_process/app_main.cpp:308-343（节选）

```
if (startSystemServer) {
args.add(String8("start-system-server")); // 传给 ZygoteInit
}
...
if (zygote) {
runtime.start("com.android.internal.os.ZygoteInit", args, zygote);
} else if (className) {
runtime.start("com.android.internal.os.RuntimeInit", args, zygote);
}
```

`AndroidRuntime::start`（同目录 `AndroidRuntime.cpp`）依次 `startVm`（创建 ART 虚拟机实例）→ `startReg`（注册数组、视图、Binder 等 JNI 方法）→ 反射调用 `ZygoteInit.main`——**native 世界把接力棒交给 Java 的准确坐标就是这一行反射调用**。

## 6.2 ZygoteInit.main：五大乐章

`ZygoteInit.java:821-938` 的结构极其工整：

java frameworks/base/core/java/com/android/internal/os/ZygoteInit.java:821-938（骨架）

```
public static void main(String[] argv) {
ZygoteHooks.startZygoteNoThreadCreation(); // :826 禁止起线程（fork 安全前提）
Os.setpgid(0, 0); // :830 独立进程组
...
```

```
if (!enableLazyPreload) {
preload(bootTimingsTraceLog); // :889 乐章①：预加载
}
gcAndFinalize(); // :897 乐章②：压实堆
Zygote.initNativeState(isPrimaryZygote); // :902 乐章③：native 状态（USAP 池 fd 等）
zygoteServer = new ZygoteServer(isPrimaryZygote); // :906 乐章④：创建 /dev/socket/zygote
if (startSystemServer) {
Runnable r = forkSystemServer(...); // :909 乐章⑤：fork system_server
if (r != null) { r.run(); return; } // 子进程直接跑 SystemServer.main
}
caller = zygoteServer.runSelectLoop(abiList);// :923 父进程：孵化循环（永不返回）
}
```

preload 的清单（:134-173）：preloadClasses（逐行实例化 /system/etc/preloaded-classes 里的三千多个类，源头是 frameworks/base/preloaded-classes）→ cacheNonBootClasspathClassLoaders → preloadResources（常用 drawable/字符串）→ nativePreloadAppProcessHALs → maybePreloadGraphicsDriver → 共享库/文字资源 → WebViewFactory.prepareWebViewInZygote。这份清单是 AOSP 用开机 trace 长期调优的产物——**多预载浪费内存，少预载首帧变慢**，它是一个全机级的权衡解。

## 6.3 前奏：preload 源码与清单哲学

java ZygoteInit.java:134-173（骨架）

```
static void preload(TimingsTraceLog bootTimingsTraceLog) {
beginPreload(); // ART 钩子：进入预加载态
bootTimingsTraceLog.traceBegin("PreloadClasses");
preloadClasses(); // ① 三千余类逐个 Class.forName
cacheNonBootClasspathClassLoaders();
preloadResources(); // ② drawable/字符串缓存
nativePreloadAppProcessHALs(); // ③ HAL 预热
maybePreloadGraphicsDriver(); // ④ 图形驱动预映射
preloadSharedLibraries(); // ⑤ System.loadLibrary 常驻
preloadTextResources();
WebViewFactory.prepareWebViewInZygote(); // ⑥ WebView 预备
endPreload();
warmUpJcaProviders(); // ⑦ 加密提供者预热
sPreloadComplete = true;
}
```

清单文件 frameworks/base/preloaded-classes 由 AOSP 的 frameworks/base/tools/preload 工具从真实开机 trace 挖掘而来（类加载计数超阈值者入选）。它是一份**活文档**：新版本会增删条目，厂商也会按机型裁剪。读它的正确姿势：随便挑一行类名，在 framework 源码里找到它，思考“为什么所有进程都需要它预载”——这份清单就是“Android 框架最内核工作集”的定义。

## 6.4 forkSystemServer：一个进程的出生证明

java frameworks/base/core/java/com/android/internal/os/ZygoteInit.java:711-789（节选）

```
/* 硬编码的出生证明 */
String[] args = {
// 讲解注释①：uid/gid 1000=system 用户；groups 是一串系统组
```

```
// （1001=phone、3003=inet 等）——system_server 的"社交圈"在此写死。
"--setuid=1000", "--setgid=1000",
"--setgroups=1001,1002,1003,1004,1005,1006,1007,1008,1009,1010,1018,1021,"
+ "1023,1024,1032,1065,3001,3002,3003,3005,3006,3007,3009,3010,3011,3012",
// 讲解注释②：能力位图双份（permitted/effective）——CAP_KILL、CAP_NET_ADMIN
// 等 13 项（:685-699），出生即最小权限白名单。
"--capabilities=" + capabilities + "," + capabilities,
"--nice-name=system_server", // 讲解注释③：进程名（ps 可见）
"--runtime-args",
"com.android.server.SystemServer", // 讲解注释④：入口类（书2 第七章）
};
...
pid = Zygote.forkSystemServer(parsedArgs.mUid, parsedArgs.mGid, ...); // :770 native fork
...
if (pid == 0) { // :782 子进程侧
if (hasSecondZygote(abiList)) waitForSecondaryZygote(socketName);
zygoteServer.closeServerSocket(); // :787 关掉孵化 socket
return handleSystemServerProcess(parsedArgs); // :788 进入 SystemServer
}
```

这段“出生证明”回答了三个问题：system_server 的 uid/gid 为何是 1000（system 用户）；它为什么握有 CAP_KILL/CAP_NET_ADMIN 等 13 项能力而普通应用没有；它为什么必须由 zygote 长 fork 而不是 init 直接拉起（要继承预加载的整个 Java 世界）。

## 6.4 runSelectLoop：孵化流水线

`ZygoteServer.java:425` 起：`Os.poll` 监听服务 socket、已接会话 socket 与 USAP 池管道。新连接由 `acceptCommandPeer`（:541）接纳；会话可读则 `processCommand`（:552）解析孵化参数（uid/gid/ABI/capabilities/rlimits/seInfo…），调 `Zygote.forkAndSpecialize`（`Zygote.java:347`）。**子进程带** `Runnable` **返回**（:562）退出循环，经 `zygoteInit`（`ZygoteInit.java:980`）三步——`RuntimeInit.commonInit`、`nativeZygoteInit`（在 `app_main.cpp:92` 的 `onZygoteInit` 里 `ProcessState::self()->startThreadPool()`，**Binder 线程池在此刻诞生**，第三册详述）、`applicationInit` → `ActivityThread.main`。父进程继续守在 poll 上，等下一个孵化请求。

**USAP 池**（:438-513）：预孵化若干“未特化”进程填池，高峰期孵化只需“特化”免去 fork；池有补水水位线与延迟节流。这是 Android 10 引入的孵化优化，判断设备是否启用可看 `init.zygote64.rc` 的 `socket usap_pool_primary` 声明。

## 6.5 runSelectLoop 源码精读

java frameworks/base/core/java/com/android/internal/os/ZygoteServer.java:539-562（节选）

```
if (pollIndex == 0) {
// 讲解注释①：index 0 永远是服务 socket 本身——accept 新会话
// （客户端是 system_server 里的 ZygoteProcess）。
ZygoteConnection newPeer = acceptCommandPeer(abiList);
peers.add(newPeer);
socketFDs.add(newPeer.getFileDescriptor());
} else if (pollIndex < usapPoolEventFDIndex) {
// 讲解注释②：已接会话可读=对方写来了一次孵化请求参数串。
```

```
ZygoteConnection connection = peers.get(pollIndex);
final Runnable command = connection.processCommand(this, multipleForksOK);
// 讲解注释③：fork 之后的分水岭——mIsForkChild 为真说明"我现在是孩子"，
// 带 command 退出循环（父进程永远留在这里）。
if (mIsForkChild) {
if (command == null) throw new IllegalStateException("command == null");
return command;
}
}
```

**设计洞察 · 逐段精读：一次 fork 的"分身时刻"**

③是全书最微妙的一行：processCommand 内部调用了 forkAndSpecialize——从这一刻起**世界上有两个线程执行流**，父子各自从同一行代码继续。孩子 return command（去当 App），父亲继续 for 循环（继续当产房）。判断自己是谁的唯一依据是 mIsForkChild 标志。理解"fork 之后代码被两人各走一遍"，你就理解了 Unix 进程模型的精髓。

processCommand 内部：读出参数字符串数组（--setuid=10023 --runtime-args ... android.app.ActivityThread）→ ZygoteArguments 解析 → forkAndSpecialize（Zygote.java:347）。**native 层 fork 后的差异处理**（com_android_internal_os_Zygote.cpp）是本册最深的一段 C++：子进程要降 uid/gid、设置能力、关掉 zygote 的 socket 与不需要的 fd、挂上自己的 seinfo 标签——一个新沙盒的所有边条在两次系统调用间完成。

USAP 池的节奏（:438-513）：池空阈值触发 mUsapPoolRefillTriggerTimestamp 记时，延迟窗口过后批量 fork 补水（DELAYED 状态）；请求数高于池容时直接退化为普通 fork。它解决的是**冷启动风暴**：桌面上十几个 App 同时被点开时，fork 本身的耗时也会排队。

## 6.6 Zygote 的内存世界：为什么 fork 能省百 MB

### 预加载后的内存布局

text zygote 进程的地址空间（fork 前）

```
┌──────────────────────────────────────┐ 高地址
│ 栈（Stack） │
├──────────────────────────────────────┤
│ │
│ ↑ 空闲区域 ↑ │
│ │
├──────────────────────────────────────┤
│ 共享的只读映射（COW 的核心！） │
│ ┌──────────────────────────┐ │
│ │ framework.jar (boot.art) │ │ ← 所有 App 共享的类+机器码
│ │ framework-res.apk 资源 │ │ ← 图标/字符串/布局
│ │ preloadClasses (3000+ 类) │ │
│ │ preloadResources (drawable) │ │
│ │ preloadSharedLibraries │ │
│ └──────────────────────────┘ │
├──────────────────────────────────────┤
│ 堆（Heap，GC 管理） │ ← 每次 fork 的 COW 基线
├──────────────────────────────────────┤
```

```
| .data / .bss |
├──────────────────────────────────────┤
| .text（ART 自身代码） |
├──────────────────────────────────────┤ 低地址
// 讲解注释①："共享的只读映射"是 Zygote 的核心价值——
// fork 后子进程与父进程共享同一物理页（内核 PTE 标只读）
// 只有写入时才触发 page fault → 内核复制该页（Copy-On-Write）
// → 未写入的页永远共享 = 零内存成本
```

### fork 后的 COW 演示

text fork 后：共享 vs 私有

```
父进程（zygote）  子进程（App）
┌──────────────┐  ┌──────────────┐
| framework 类 | ═══════ | framework 类 | ← 共享（只读）
| "Button" 类 | ═══════ | "Button" 类 | ← 共享
| 堆：对象A | ✗ | 堆：对象A' | ← COW（App 改了 A）
| 堆：对象B | ═══════ | 堆：对象B | ← 共享（没改）
└──────────────┘  └──────────────┘
// 讲解注释②：App 只为它改过的对象付内存——
// 典型 App 只修改 ~10% 的堆对象
// 所以每个 App 的实际增量内存 ≈ 总堆 × 10%
// 而不是从零分配整个堆
```

### 定量估算

| 项 | 无 Zygote（每进程独立加载） | 有 Zygote（COW 共享） |
|---|---|---|
| framework 类 | 约 120 MB × N 进程 | 120 MB（一份物理页） |
| 资源（drawable/字符串） | 约 30 MB × N | 30 MB（一份） |
| ART 运行时 | 约 10 MB × N | 10 MB |
| App 自身堆 | -- | ~5-20 MB（因 App 而异） |
| **总计（10 个 App）** | **~1.6 GB** | **~200-350 MB** |

**设计洞察 · 逐段精读：这就是为什么 Android 能跑 100+ 进程**

没有 Zygote+COW，每个 App 进程独立加载 160 MB 框架世界——8GB 设备最多跑 ~40 个进程。有了 COW，100 个 App 共享同一份物理页，增量只有各 App 自己修改的部分。**Zygote 是 Android 多进程模型的"地基桩"**——没有它，"每个 App 独立进程"的安全模型在内存上就不可行。

# 第七章 SystemServer：三阶段服务总装

## 7.0 run() 前奏代码精读

java frameworks/base/services/java/com/android/server/SystemServer.java:802-899（骨架）

```
Binder.setWarnOnBlocking(true); // :802 禁同步外呼（防优先级反转）
```

```
...
BinderInternal.setMaxThreads(sMaxBinderThreads); // :854 31 线程（:435 定义）
...
Looper.prepareMainLooper(); // :860 本线程成为系统主线程
System.loadLibrary("android_servers"); // :867 加载 native 服务实现
createSystemContext(); // :882 系统 ActivityThread 与 Context
mSystemServiceManager = new SystemServiceManager(mSystemContext); // :892
LocalServices.addService(SystemServiceManager.class, mSystemServiceManager); // :897
SystemServerInitThreadPool tp = SystemServerInitThreadPool.start(); // :899 并行初始化池
```

`createSystemContext` 值得单独说：system_server 里跑着“世界上第一个 ActivityThread”，它没有界面，但拥有完整的 Context 体系——此后所有系统服务的 `mContext` 都从这里来。理解“system_server 也是一个（特殊的）应用进程”，很多 API 行为（如 registerReceiver、ContentResolver）才讲得通。

**PMS 为什么在 Bootstrap 批**：AMS 启动即需查询“哪些是 persistent 应用要提前拉起、权限如何”；WMS/Home 更依赖包扫描结果。所以顺序是 Installer→AMS（先占位）→phase 100→PMS（扫包）→……→PMS.systemReady(:2712)→AMS.systemReady(:2801)——**AMS 先生、PMS 后熟**，中间隔着整个 Other 批。

## 7.1 run() 的前奏（:761-929）

`SystemServer.java:761` 的 `run()` 在进入服务启动前完成自我装配：记录启动计数属性（:767-769）、时区与 locale 校正（:777-799）、`Binder.setWarnOnBlocking(true)`（:802，**禁止 system_server 同步调用其他进程**——防止低优先级进程反向卡死系统）、`BinderInternal.setMaxThreads(31)`（:854，Binder 线程预算翻倍）、`Looper.prepareMainLooper()`（:860）、`System.loadLibrary("android_servers")`（:867）、`createSystemContext`（:882）、new `SystemServiceManager`（:892）并放进 `LocalServices`（:897，同进程服务总线）、`SystemServerInitThreadPool.start()`（:899，并行化非关键初始化）。

## 7.2 三阶段与依赖即顺序




图 5 SystemServer 三阶段服务启动清单与 boot phase 100→1000 全局时间线，附 run() 硬核细节

**Bootstrap 批**（`:1052-1330`）的原则是“后序服务的地基”：Installer（installd 客户端）→ **AMS**（`:1122`）→ boot phase 100（等默认显示屏，`:1202`）→ **PMS**（`:1233`，扫描全机 APK）→ UserManager → OtaDexopt。**Core 批**（`:1336`）是轻量统计类：BatteryService、UsageStats、WebViewUpdate…。**Other 批**（`:1404-3059`，224 个 trace 段）是大合唱：`InputManagerService`（`:1570`）、**WMS**（`:1586`）、phase 480/500/520（`:2665/2669/2756`）、`PMS.systemReady`（`:2712`）、`AMS.systemReady`（`:2801`）、`startSystemUi`（`:3053`）。

`SystemServiceManager.startService`（`SystemServiceManager.java:207`）用反射调 `new Service(Context)` 构造服务，再调其 `onStart()`；`startBootPhase`（`:279`）向所有已启动服务广播 phase，服务在 `onBootPhase(phase)` 里做阶段挂钩（如 AMS 等到 550 才放行三方调用）。

## 7.3 Bootstrap 批完整清单（traceBegin 顺序）

| 顺序 | 服务 | 一句话职责 |
|---|---|---|
| 1 | Watchdog(:1058) | 死锁看门狗（先行注册才能监视后序） |
| 2 | SystemConfig | 读取 system/etc/sysconfig 全局配置 |
| 3 | PlatformCompat | 平台兼容性框架（AMS/PMS 都依赖） |
| 4 | FileIntegrityService | fs-verity 文件完整性 |
| 5 | **Installer** | installd 客户端（dexopt/安装执行） |
| 6 | DeviceIdentifiersPolicyService | 设备标识合规 |

| 顺序 | 服务 | 一句话职责 |
| --- | --- | --- |
| 7 | UriGrantsManagerService | URI 授权 |
| 8 | PowerStatsService / IStats / Memtrack | 能耗与统计 |
| 9 | **ATM+AMS**(:1122) | 调度中枢占位（setInstaller 关联：1124+） |
| 10 | DataLoaderManager / IncrementalService | 增量安装支撑 |
| 11 | PowerManagerService(:1147) | 电源策略 |
| 12 | ThermalManagerService / HintManager | 温控与性能暗示 |
| — | boot phase 100(:1202) | 等待默认显示屏 |
| 13 | **PackageManagerService**(:1233) | 全机 APK 扫描与权限基线 |
| 14 | UserManagerService | 多用户 |
| 15 | OtaDexoptService(:1266) | OTA 后的 dex 整理（后台化） |

这张表是“依赖即顺序”的具象化：每一行都在为后面某行扫清障碍。

## 7.4 服务启动顺序的因果链详解

这张表解释了"为什么必须是这个顺序"——每行都是一条因果链。

### Bootstrap 批内部的严格序列

text Bootstrap 批的 12 步依赖链（不可调换）

```
1. Watchdog
↑ 为什么第一：后序服务要注册到 Watchdog 监控死锁
2. PlatformCompat
↑ 为什么第二：AMS/PMS 都要查兼容性行为
3. Installer
↑ 为什么第三：AMS 安装系统包需要 dexopt → installd 客户端
4. ATM + AMS (:1122)
↑ 为什么第四：后序服务的组件调度都靠 AMS
| setInstaller 关联第 3 步的 Installer
5. boot phase 100（等默认显示屏）
↑ 为什么在此：AMS 要知道主屏尺寸才能调度 Activity
6. PMS (:1233)
↑ 为什么第五：AMS 要查"哪些 persistent 应用要提前拉起"
| WMS/Home 更依赖包扫描结果（Launcher 是 APK！）
7. UserManagerService
↑ 为什么第六：多用户元数据
8. OtaDexoptService
↑ 为什么最后：OTA 后的 dex 整理是后台任务
// 核心洞察：顺序不是随意的，是"依赖拓扑排序"——
// 后序服务构造器里直接调先序服务的接口，乱序=NullPointerException
```

### 为什么 Core 批不和 Bootstrap 合并

Core 批的服务（BatteryService、UsageStatsService 等）**没有重依赖**（不需要等 Installer/AMS/PMS），但也不是"无足轻重"——把它们的启动与 Bootstrap 并行执行（SystemServerInitThreadPool），可以缩短 wall time。**拆分的本质是并行化机会的边界**。

### startOtherServices 的 224 段为何最长

Other 批从 `:1404` 到 `:3059` 跨越 1600+ 行——因为 200+ 服务各自有初始化条件、相互依赖、boot phase 挂钩、异常处理。**这是 SystemServer.java 最大的函数**，也是系统开发者的"日常阅读主场"。

> **设计洞察 · 逐段精读：面试终极串联题**
>
> "说出从按下电源键到 system_server 就绪的完整链"——这个答案就是本册全部内容。但加分项是**因果**："为什么 Installer 在 AMS 之前？因为 AMS 启动就要装系统包，没有 dexopt 客户端寸步难行。"——能说出因果链的候选人才真正理解了系统。

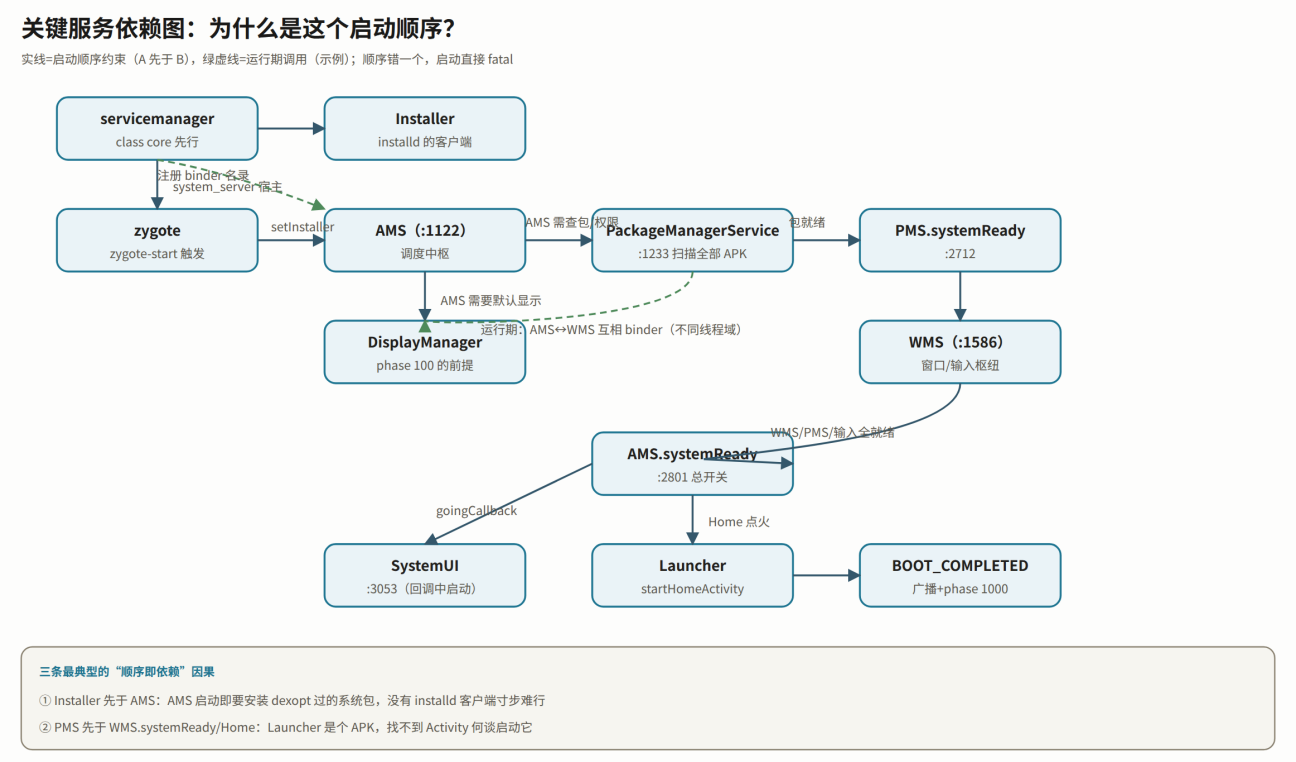



图 6 关键服务依赖图：servicemanager/zygote/Installer/AMS/PMS/WMS 的启动顺序因果与运行期调用

# 第八章 开机完成：systemReady 到 BOOT_COMPLETED

## 8.0 systemReady 内部一览

java ActivityManagerService.java:8120-8144（节选）

```
t.traceBegin("retrieveSettings");
retrieveSettings(); // 回读全局设置
...
if (goingCallback != null) goingCallback.run(); // :8141 SystemServer 的回调在此执行
final int currentUserId = mUserController.getCurrentUserId();
...
t.traceBegin("ActivityManagerStartApps");
mBatteryStatsService.onSystemReady(); // 统计就位
...（后续：启动 persistent 应用、phase 550/600）
```

回调里 SystemServer 做的事（SystemServer.java:2801-3061）：通知 WindowManager“系统正在就绪”、启动 SystemUI（:3053）、开启 watchdog 正式监控、各服务 systemReady 链式传播。**两段式设计的动机**：AMS 的就绪依赖大量外部状态（包扫描、用户解锁、显示就绪），回调让“通知方”与“被通知方”解耦——AMS 不需要知道谁需要被叫醒。

## 8.1 systemReady 的两段式

ActivityManagerService.systemReady（ActivityManagerService.java:8039）先做内部就绪（清理临时进程、准备 user），goingCallback.run()（:8141）执行 SystemServer 传来的回调（SystemServer.java:2801-3061：启动 SystemUI、通知各服务收尾），然后按序 phase 550/600。Launcher 的点火链：ATMS.startHomeActivity（ActivityTaskManagerService.java:5874）→ RootWindowContainer.java:1493 → ActivityStartController.startHomeActivity——**Home 是第一个被启动的“应用”**。

## 8.2 finishBooting：动画是最后一道闸门

java frameworks/base/services/core/java/com/android/server/am/ActivityManagerService.java:5067-5086（节选）

```
final void finishBooting() {
synchronized (this) {
if (!mBootAnimationComplete) { // :5075 动画未完，记账后先返回
mCallFinishBooting = true;
return;
}
}
ZYGOTE_PROCESS.bootCompleted(); // :5085 通知 zygote（USAP 统计等）
VMRuntime.bootCompleted(); // :5086 通知 ART 进入“开机后”策略
...
SystemProperties.set("sys.boot_completed", "1"); // :5149 全机权威标志
```

开机动画退出 → bootAnimationComplete（:5240-5248）置位并补调 finishBooting()——**动画是用户可感知的最后一道闸门**，代码用 mBootAnimationComplete 门闩保证“广播必在动画之后”。随后 ACTION_BOOT_COMPLETED 有序广播发出、phase 1000 推进，接收方从三方 App 到 telephony 依次被唤醒。

## 8.3 BOOT_COMPLETED 的完整时序（教科书级逐步分解）

BOOT_COMPLETED 不是一条广播——它是一串事件的**终点**。以下是完整时序（含每步的系统属性变化）。

### 阶段 A：systemReady 之前（AMS 还没准备好）

```text
t0: SystemServer startOtherServices 执行到 :2800 附近
│
├→ 所有 Bootstrap/Core/大部分 Other 服务已启动
├→ PMS.systemReady(:2712) 已调用
│ └→ 包扫描完成，所有 APK 可查询
│
└→ mActivityManagerService.systemReady(goingCallback) :2801
│
├→ goingCallback.run() :8141
│ └→ SystemServer 的回调：
│ ├→ 恢复设置（retrieveSettings）
│ ├→ 通知各服务"系统即将就绪"
│ └→ 启动 SystemUI（startSystemUi :3053）
│
└→ 继续后续启动流程...
```

### 阶段 B：Home 点火

```text
t1: AMS 启动 Launcher
│
├→ ATMS.startHomeActivity (ActivityTaskManagerService.java:5874)
│ └→ RootWindowContainer.java:1499
│ └→ ActivityStartController.startHomeActivity
│ └→ "Home 是第一个被启动的应用"（书4 §一）
│
├→ Launcher 进程不存在 → ProcessList.startProcessLocked
│ └→ zygote fork → ActivityThread.main → onCreate
│
└→ Launcher 显示桌面 → 用户看到"桌面"
```

### 阶段 C：动画退出与最终标志

```text
t2: bootanimation 退出
│
├→ bootanimation 是独立进程（class core animation）
├→ 退出条件：service list 中出现 "surfaceflinger" 且 sys.boot_completed
│ 实际机制：PropertyOverlay 在 SF 侧监控 bootanim 状态
│
└→ bootAnimationComplete 回调到 AMS (:5240-5248)
└→ mBootAnimationComplete = true
└→ 若 mCallFinishBooting = true → 补调 finishBooting()
```

```text
```

```
t3: finishBooting (:5067)
|
├→ 动画门闩检查 (:5075-5083)
| if (!mBootAnimationComplete) { mCallFinishBooting = true; return; }
| // 动画没完就先记账等回调——"动画是最后一道闸门"
|
├→ ZYGOTE_PROCESS.bootCompleted() (:5085)
| └→ 通知 zygote：开机完成（USAP 统计等）
|
├→ VMRuntime.bootCompleted() (:5086)
| └→ 通知 ART：切换到"开机后"GC 策略
|
├→ SystemProperties.set("sys.boot_completed", "1") (:5149)
| └→ ★ 全机权威标志！★
| └→ init 检测到此属性 → 触发 on property:sys.boot_completed=1 的 action
| └→ 通知各服务做开机后调整
|
└→ 发送 ACTION_BOOT_COMPLETED 有序广播
└→ 接收者按优先级依次收到（RECEIVE_BOOT_COMPLETED 权限）
└→ 最后 → SystemService.PHASE_BOOT_COMPLETED (1000)
```

### 阶段 D：用户可感知的"开机完成"

text

```
t4: 用户感知
|
├→ 桌面已显示 (t1 的结果)
├→ 动画已退出 (t2 的结果)
├→ BOOT_COMPLETED 广播已送达 (t3 的结果)
|
└→ "开机完成" = 三个事件全部到位
时间上：桌面显示 < 动画退出 < BOOT_COMPLETED
（桌面可能比 BOOT_COMPLETED 早数秒）
```

### 关键属性的时间线

| 属性 | 设置时机 | 含义 | 谁消费 |
|---|---|---|---|
| sys.boot_completed=1 | finishBooting :5149 | 框架就绪 | init 触发 on property |
| dev.bootcomplete=1 | finishBooting 尾部 | 设备级完成 | telephony 等 |
| init.svc.zygote=running | Service::Start :684 | 服务状态机 | 运维工具 |
| ro.runtime-init | SystemServer run() :767 | 启动计数 | 性能统计 |

> **设计洞察 · 逐段精读：为什么 BOOT_COMPLETED 由 AMS 而非 init 设置**
>
> "系统可用"的判据是**框架就绪**（Home 可启动、广播可送达）而不是"服务都在跑"——只有 AMS 有这个全局视角。init 只知道"服务起了"，不知道"服务能不能工作"。这就是为什么 sys.boot_completed 的设置者在 AMS.java:5149 而不是 init.cpp。

**源码锚点 · 第八章锚点速查**

<font face="MonoFT" size="9">SystemServer.java:2801</font> mActivityManagerService.systemReady(回调)<br/><font face="MonoFT" size="9">ActivityManagerService.java:8039</font> systemReady 定义 / <font face="MonoFT" size="9">:8141</font> goingCallback.run<br/><font face="MonoFT" size="9">ActivityTaskManagerService.java:5874</font> startHomeActivity / <font face="MonoFT" size="9">RootWindowContainer.java:1493</font> 点火<br/><font face="MonoFT" size="9">ActivityManagerService.java:5067</font> finishBooting / <font face="MonoFT" size="9">:5075</font> 动画门闩 / <font face="MonoFT" size="9">:5149</font> sys.boot_completed=1

# 第九章 启动性能：主线上的优化战场

读完白盒链路，性能话题就有了坐标。启动优化在 AOSP 里是持续十几年的工程，本册给出现代方法的骨架：

**度量为先**。`bootchart`（init 内建）画甘特图定位“哪个服务占了哪段时间”；`logcat -b events` 的 `boot_progress_*` 事件（preload_start/end、system_run 等）给出阶段锚点；perfetto（`external/perfetto`）可叠加 CPU/IO/调度视角。**没有度量的启动优化都是玄学**——本册所有行号锚点同时也是埋点位置。

三个经典优化位：

- **预加载裁剪**：`preloaded-classes` 每行一个类，低内存设备（Go 版）走 `--enable-lazy-preload`（`ZygoteInit.java:855`）延迟到首 fork 前；类清单增删直接改变全机内存水位——这是“内存-首帧”权衡的最前线。
- **并行化**：SystemServer 的非关键初始化进 `SystemServerInitThreadPool`（`:899`）；init 侧 ueventd 与主流程并行冷插；APEX 激活（apexd）在 post-fs-data 与服务启动重叠。
- **孵化加速**：USAP 池削 fork 峰值；zygote 的 `gcAndFinalize`（`:897`）减小 COW 复制基数；profile 引导的 ART 预编译让 system_server 的类在 boot.art 里就是机器码。

一个反直觉的事实收尾：**开机最贵的单项常常是 PMS 扫描全部 APK**（`:1233`）与预加载资源——它们都是“用一次性的贵换之后每次的便宜”。理解启动优化，本质是理解 Android 在“首次成本”与“稳态成本”之间做的所有交易。

# 第十章 设计决策十问

本章把主线上的十个“为什么”整理成问答，每条都可直接追溯到前文锚点。

- **为什么 init 单线程？** 无锁、公平、确定性；代价是吞吐，但启动期没有吞吐需求（第五章）。
- **为什么两阶段 init + 两次 re-exec？** 根文件系统与 SELinux 的鸡生蛋问题，用“每阶段只信任自己布置的世界”解决（第二章）。

- **为什么 rc 是声明式语言？** 同一语法服务开机求值、构建期校验、CTS 检查三处；静态可分析是系统软件的刚需（第四章）。
- **为什么先 reap 再处理其他 fd 事件？** 消除“服务已死未收尸、ctl.start 被拒”竞态（第五章，`init.cpp:1025-1028` 注释）。
- **为什么属性先行？** 触发器、服务的 SELinux 上下文、加密状态全部依赖属性区存在（第三章）。
- **为什么 zygote 禁止起线程？** fork 多线程进程会复制出不一致的锁状态，POSIX 只保证 fork 方线程存活；`startZygoteNoThreadCreation`（`ZygoteInit.java:826`）从源头杜绝。
- **为什么 fork 而不是 exec 应用？** exec 会丢掉预加载的类/资源/ART 状态；fork+写时复制让“加载一次、千次复用”（第六章）。
- **为什么 system_server 由 zygote 第一个 fork 而非 init 拉起？** 它需要整个 Java 框架世界；同时它出生即带 13 项能力、1000 组身份——最小权限在出生时锁定（6.3 节）。
- **为什么服务分三批+boot phase？** 依赖拓扑必须显式化；phase 是全局进度条，服务按阶段就位避免“查无此人”（第七章）。
- **为什么 bootanimation 独立进程？** 它要在 SF 就绪而 AMS 未就绪的窗口里播动画；死了不该影响开机（class core animation 但非 critical）。
- **为什么 SystemUI 由回调启动而不是 rc？** 它是“应用”（有 UI、依赖包扫描与权限），必须在 PMS/AMS 就绪后；rc 世界管不了 Java 依赖。
- **为什么 sys.boot_completed 由 AMS 设置而非 init？** “系统可用”的判据是**框架就绪**（Home 可启动、广播可送达）而不是服务跑起来；只有 AMS 有这个视角（第八章）。

# 第十一章 深水区：bootloader、fastboot 协议与 OTA 引擎

## 11.1 bootloader 的两级跳

现代设备两级引导：**первич bootloader（芯片固化 SPL）→ avb 校验二级 bootloader（aboot/ABL，bootable/ 的各平台实现）**→ ABL 读 boot header（书1 §7.1 的 v4）拉内核。ABL 的决策输入：BCB（misc 分区的 boot control block，recovery/ota 想干什么写在里面）+ 按键组合 + charger 状态；输出：normal/bootloader/recovery/fastboot 四种去向。**开机键的"长按进 fastboot"本质是 ABL 在验签前后的一个轮询窗口**。

## 11.2 fastboot 协议最小集（host 侧源码：system/core/fastboot）

text fastboot 会话（USB bulk 传输，每命令-响应一问一答）

```
host: getvar:max-download-size → 探测可下载块大小
host: download:0x...<data> → 分块推镜像到设备 RAM
host: flash:boot_a → 写入当前槽的 boot 分区
host: getvar:current-slot → A/B 槽位寻址（slot 后缀 _a/_b）
host: set_active:b → 切换启动槽（boot_control HAL 兑现）
```

```
host: reboot[-bootloader|fastboot|recovery]
```

fastbootd（userspace 变体，走 adbd 通道）存在的唯一理由：**动态分区**。bootloader 看不懂 super 容器的 metadata，逻辑分区（system/vendor/…）必须由跑起来的系统代刷——这就是"刷 GSI 要先 `fastboot reboot fastboot`"的机制解释。

## 11.3 GKI 与 boot config

GKI（通用内核镜像）= 内核与 vendor 解耦的镜像版 Treble：boot 分区只装 Google 签名的通用内核，厂商差异走 vendor_boot（vendor ramdisk）、vendor_dlkm（内核模块）与 **bootconfig**（内核 cmdline 的结构化继任者，`androidboot.*` 参数家族）。开机链新增一步：内核启动时读 bootconfig 分区/区域合并参数——排查"参数没生效"时先分清 cmdline 还是 bootconfig 域。

## 11.4 update_engine：OTA 的执行引擎

`system/update_engine/` 的骨架是**动作流水线**（ActionProcessor 逐 Action 执行）：InstallPlanAction（解 payload manifest）→ FilesystemVerifierAction（校验旧版哈希）→ DownloadAction（HTTP 拉包+DeltaPerformer 增量还原：源块搬运/bz4 解压→写后台槽）→ PostinstallRunnerAction（新槽的OTA脚本）→ boot_control 切槽。失败语义：任意 Action 失败即整单回退，旧槽未动一根毫毛。**payload 签名验两次**（下载前 manifest 签名、块级 hash 树）——与 AVB 构成三明治防御。

> **源码锚点 · 本章锚点**
> <font face="MonoFT" size="9">system/core/fastboot/</font>：host 工具与 vendor_boot_img 处理<br/><font face="MonoFT" size="9">bootable/libbootloader/libxbc</font>：bootloader 消息控制库<br/><font face="MonoFT" size="9">system/update_engine/payload_consumer/</font>：DeltaPerformer 增量还原<br/><font face="MonoFT" size="9">build/make/core/Makefile:846-985</font>：dtb/boot 镜像产物规则（书1 §7）

# 附录 A 观测实操手册

| 目标 | 命令 | 对应源码 |
| --- | --- | --- |
| 看两阶段切换 | dmesg 检索 init | init.cpp:839 "init second stage started!" |
| 看触发瀑布 | getprop init.svc.* 的时序 | service.cpp:684 NotifyStateChange |
| 看预加载 | logcat -b events 过滤 preload | ZygoteInit.java:887-890 |
| 看服务启动 | logcat 过滤 SystemServer | SystemServer.java 各 traceBegin |
| 看 boot phase | dumpsys activity service SystemUIService 等 | SystemServiceManager.java:279 |
| 权威开机标志 | getprop sys.boot_completed | AMS.java:5149 |

| 目标 | 命令 | 对应源码 |
| --- | --- | --- |
| 启动时间线 | bootchart / perfetto | system/core/init/bootchart.cpp |
| 孵化耗时 | logcat -b events 过滤 am_proc_start | Process.start → zygote socket |

# 附录 B 关键文件清单（本册出场人物）

| 文件 | 行数 | 角色 |
| --- | --- | --- |
| system/core/init/main.cpp | 83 | 四身份入口分发 |
| system/core/init/first_stage_init.cpp | 400+ | 挂分区、切根 |
| system/core/init/init.cpp | 1062 | 第二阶段主体与主循环 |
| system/core/init/parser.cpp | 246 | rc 解析器框架 |
| system/core/init/action_manager.cpp | 110+ | 事件队列与求值 |
| system/core/init/builtins.cpp | 1400+ | 全部内置命令 |
| system/core/init/service.cpp | 871 | 服务生命周期 |
| system/core/init/property_service.cpp | 1396 | 属性仲裁 |
| system/core/rootdir/init.rc | 1300+ | 开机总谱 |
| frameworks/base/cmds/app_process/app_main.cpp | 344 | zygote 壳与 JNI 桥 |
| core/java/com/android/internal/os/ZygoteInit.java | 1006 | Java 世界第一入口 |
| core/java/com/android/internal/os/ZygoteServer.java | 600+ | 孵化循环 |
| services/java/com/android/server/SystemServer.java | 3216 | 服务总装线 |
| services/core/.../SystemServiceManager.java | 300+ | 服务生命周期管理 |
| services/core/.../am/ActivityManagerService.java | 18480 | 调度中枢（本册只涉启动切片） |

# 附录 C 自检题

- 说出 init 的四个身份及触发条件（第二章）。
- `on late-init` 的子触发器顺序为何不能乱？调换 `post-fs-data` 与 `zygote-start` 会怎样？（第四章，提示：/data 加密与 dex）
- zygote 为什么先 `gcAndFinalize()` 再 fork？（第六章，提示：COW 复制的页数）
- system_server 的 Binder 线程上限是多少、在哪个文件哪个常量？（第七章：SystemServer.java:435）
- `sys.boot_completed` 与 `dev.bootcomplete` 谁先谁后、各代表什么？（第八章与附录 A）

# 附录 D 动手实验（一天量）

实验一“时序考古”：`adb shell dmesg | grep -E "init|zygote"` 找到 "init second stage started!"（init.cpp:839）；`logcat -b events` 过滤 boot_progress_ 事件，把时间戳画成时间线，与本册总时序图对位。

实验二“状态机观察”：每秒一次 `getprop init.svc.zygote` 与 `init.svc.surfaceflinger`，重启设备观察 running→restarting→running 的流转（对应 service.cpp:684 NotifyStateChange 与 Reap :258）。

实验三“剧本推演”：打开 rootdir/init.rc，从 late-init（:516）逐 trigger 朗读子触发器；假设你要加一个“post-data 阶段启动的自研守护进程”，写出它的 rc 片段（service 段 + 放哪个 etc/init 目录 + SELinux 三件套），对照第四章 4.5 的语法表自查。

实验四“bootchart”：按 system/core/init/bootchart.cpp 注释开启采集，导出 PNG，找出占比最大的三段，分别映射到本册的哪个阶段。

实验五“思想实验”：把 zygote-start 移到 post-fs 之前会发生什么？写出三条因果链（提示：/data 加密、preloaded-classes 可达性、dex 位置）。

# 教科书习题集（分级）

## 第一级：简答题（检验概念理解）

1. init 的四个身份是什么？各由什么触发？

2. 为什么 init 要分两阶段执行？每阶段做什么？

3. rc 语言的两种触发器是什么？各在什么时候触发？

4. Zygote 为什么禁止起线程？这个约束解决了什么问题？

5. forkSystemServer 传了哪些"出生证明"参数？

6. SystemServer 为什么分三批启动服务？每批的特点是什么？

7. boot phase 的 550 和 600 分别放行什么？

8. BOOT_COMPLETED 和 sys.boot_completed 谁先谁后？各代表什么？

## 第二级：分析题（检验机制理解）

9. **追踪时序**：画出从 kernel 拉起 init 到 zygote socket 可用的时间线，标出每个阶段的关键属性变化。

10. **COW 分析**：fork 后 App A 和 App B 同时使用 Button 类的 onClick 方法——它们的 onClick 代码物理上是同一份内存吗？为什么？

11. **失败推演**：如果 init.rc 的 late-init 触发链中 zygote-start 被移到 post-fs 之前，会发生什么？给出三条因果链。

12. **并发对照**：init 是单线程事件驱动，system_server 是几十个 binder 线程+Handler 线程池——为什么两个进程选择了截然不同的并发模型？

13. **顺序依赖**：如果 PackageManagerService 在 AMS 之前启动（调换顺序），哪个服务会先崩溃？为什么？

## 第三级：设计题（检验系统思维）

14. **设计决策**：假设 Google 要把 preload 从"全量预载"改为"按需预载+云端 profile"（Android 12+ 的方向），你会怎么设计这个系统的接口？（提示：profile 的生成/下载/应用/回退）

15. **性能方案**：一台低端设备（1GB 内存）开机 40 秒，你要在两周内优化到 25 秒，给出分阶段方案（第一周度量+定位，第二周定向优化）。

16. **架构演进**：如果要支持"system_server 运行时重启不重启 zygote"（Android 已在做），需要改动哪些机制？列出三个必须解决的核心问题。

## 参考答案要点

**1.** ①默认 FirstStageMain ②subcontext ③selinux_setup ④second_stage；由 argv[1] 和 argv[0] 区分。

**10.** 是同一份物理内存——onClick 是方法代码（存在 boot.art 的只读映射中），COW 页未被写入 → 共享。只有 App 各自的 Button 实例（堆对象）是独立的。

**11.** ①zygote 要读 /data 的 dex/资源 → post-fs 未完成=不可用 ②ro.crypto.state 未定=无法选触发分支 ③apexd 未激活=ART APEX 不可用。

**15.** 度量：bootchart+logcat boot_progress 分段。定位：zygote preload 慢（低端设备 IO 慢）→ 裁剪+lazy preload。system_server 慢 → PMS 并行+减 priv-app 数量。动画 → 提前放行。

# 术语表（按拼音排序）

| 术语 | 所在册 | 解释 |
| --- | --- | --- |
| APEX | 1/2 | 可独立升级的模块容器，apexd 在 post-fs-data 挂载 |
| BCB | 2 | Boot Control Block（misc 分区，bootloader 读它决定启动模式） |
| boot.img | 2 | kernel + ramdisk（GKI 后仅 kernel） |
| boot phase | 2 | SystemServiceManager 广播的全局进度条（100→1000） |
| bootchart | 2 | init 内建启动时间线采集器 |
| COW | 2 | Copy-On-Write：fork 后共享页，写时才复制 |
| critical | 2 | rc 服务属性：死则整机重启 |
| dtbo | 2 | 设备树覆盖分区（板级差异） |
| GKI | 2 | Generic Kernel Image：boot 只装 Google 签名内核 |
| init.rc | 2 | init 的配置文件（声明式语言） |
| Launcher | 2 | 桌面 App（第一个被启动的"应用"） |
| lazy preload | 2 | Go 版默认：首 fork 前才加载 preload 清单 |
| preload | 2 | Zygote 预加载的类/资源/共享库 |
| preloadClasses | 2 | frameworks/base/preloaded-classes：3000+ 类清单 |
| re-exec | 2 | 进程重新执行自身（init 两阶段的切换机制） |
| runSelectLoop | 2 | Zygote 的孵化主循环（poll + fork） |
| switch_root | 2 | 第一阶段 init 切换根文件系统 |
| system_server | 2 | 系统服务宿主进程（由 zygote fork） |
| trigger | 2 | rc 语言的事件触发器（on <trigger>） |
| USAP | 2 | UnSpecialized App Process 池（预 fork 备用进程） |
| vbmeta | 2 | AVB 的信任根分区 |

| 术语 | 所在册 | 解释 |
| --- | --- | --- |
| Zygote | 2 | App 之母进程（预加载 + COW fork） |

# 排障手册：启动问题定位

## 开机卡在 logo

1. dmesg 看内核是否完成初始化（挂载/驱动）
2. logcat 看是否进入 init 第二阶段
3. 检查 /proc/cmdline（bootconfig 参数是否正确）

→ 深入：本册第二章（第一阶段）+ 第三章（第二阶段）

## 开机卡在动画

1. logcat -b events 看 boot_progress 到了哪
2. dumpsys activity 看 AMS 是否 systemReady
3. 检查 PMS 是否完成扫描（dumpsys package | head）

→ 深入：本册第七/八章

## 开机后黑屏（Launcher 不出来）

1. dumpsys window 看是否有窗口
2. dumpsys activity activities 看 Home Activity 状态
3. 检查 Launcher APK 是否正常（pm list packages | grep launcher）

→ 深入：书5（窗口体系）+ 书4（Activity 启动）

## zygote 反复重启

1. getprop init.svc.zygote（restarting=崩溃循环）
2. logcat -s zygote 看崩溃日志
3. 检查 ART 相关（boot.art 是否损坏）

→ 深入：本册第六章

## 启动慢

1. bootchart 定位最耗时阶段
2. logcat -b events 逐 boot_progress 段计时
3. dumpsys activity services | grep "startup"

→ 深入：本册第九章（性能分析）